Frontend Dev — Lessons Learned (NutriPlan project)

A running list of real bugs I hit and the concept behind each one. Read before starting any new project.

---

1. Module imports/exports

Mistake: `import * as uiLogic from "./file.js"` then calling `showFirstLoadSections()` directly (unqualified). **Concept:** `import * as X` puts every export under the `X.` namespace — you must call `X.functionName()`. If you want to call functions unqualified, use named imports instead: `import { functionName } from "./file.js"`.

Mistake: Trying to reassign an imported variable from outside the module that declared it (`logMealObject = ""` in another file). **Concept:** Imported bindings are **read-only** in the importing file. Only the exporting module can reassign its own `export let` variable. If another file needs to reset it, export a function that does the reassignment internally (e.g. `export function clearLogMealObject(){ logMealObject = ""; }`).

Mistake: `export let mealsArray = []; mealsArray = showMeals;` — assigned a function reference instead of data. **Concept:** `export let` creates a **live binding** — when the exporting module reassigns the variable, every importer sees the update automatically. Use this to share "current state" (like fetched API data) between modules: store the real data in the variable, not a function.

---

2. Functions that are exported but never called

Mistake: Writing and exporting a function (`initNavigation`, `logMeal`, `searchProductsByBarcode`, etc.) but forgetting to actually call it in `main.js`. **Concept:** This causes **silent failure** — no console error, because no code ever runs. If a button "does nothing" and there's zero console output, the first thing to check is: is the setup function that attaches this listener actually being called somewhere?

---

3. DOM elements & event listeners

Mistake: `document.getElementById("some-dynamic-id")` called for an element that's created later via `innerHTML` (e.g. grabbing a modal's close button before the modal HTML has been inserted). **Concept:** Any element built from a template string only exists in the DOM **after** the line that sets `.innerHTML = template` runs. Any `getElementById`/`querySelector` targeting something inside that template must come strictly *after* that assignment.

Mistake: Trying to attach a listener directly to a dynamically-created, repeatedly-recreated element (`#log-meal-btn`, one card among many `.product-card`s). **Concept — Event delegation:** attach the listener once to a **stable parent** that always exists (e.g. `#log-btn`, `#products-grid`), and inside the handler use `e.target.closest(".target-class")` to detect which child was actually clicked. This survives the children being destroyed/recreated by `innerHTML` re-renders, and doesn't require re-attaching listeners after every render.

Mistake: Reading a data attribute off `e.target` directly instead of off the result of `.closest()`. **Concept:** `e.target` is whatever the user's cursor literally landed on (could be an inner `<img>`, `<span>`, icon, etc.) — not necessarily the element carrying your `data-*` attribute. Always read attributes off the `.closest(".card-class")` result, not

`e.target`.

---

4. Arrays and loops

Mistake: `for(let i = 0; i < searchMeals.length; i++)` then checking `if (searchMeals.length === 0)` **inside** the loop body. **Concept:** If `.length` is `0`, the loop condition (`0 < 0`) is false immediately — the body never runs even once, so any "empty state" check inside the loop is dead code. Check for empty **before** the loop starts, not inside it.

Mistake: `mealsArray[i].ingredients.length` fixed, but then still writing `mealsArray[i].ingredients.measure` (missing `[y]`) inside the inner loop. **Concept:** Fixing the loop bound doesn't fix property access — every reference to the current item inside a loop needs the index (`array[i]`, or `[y]` for a nested loop), not just the array/outer object.

Mistake: `${searchProducts.name}` inside a `for(i...)` loop instead of `${searchProducts[i].name}`. **Concept:** The loop variable exists so you can access *one item* at a time — always index into the array with it. Reading a property directly off the array itself (which has no `.name`, `.image`, etc.) returns `undefined` for every card.

---

5. Scoping (`let`/`const`)

Mistake: `let logModal = ...` declared inside an `if` block, then referenced outside that block a few lines later. **Concept:** `let`/`const` are **block-scoped** — the variable only exists between the `{` and `}` it was declared in. Referencing it outside throws `ReferenceError`. If you need the value after the block, declare it before the block and assign inside, or move the usage inside the block itself.

Mistake: Declaring `let mealIdNumber` (or similar) inside a callback,

then trying to use it one line *above* where it's declared/attached. **Concept:** Declare a variable in the outer scope if it needs to be used both when attaching a listener and inside the listener's callback — otherwise the callback's inner declaration isn't visible outside it.

---

6. Async / Await

Concept — what they actually do:

- `async function` marks a function as always returning a **Promise** — calling it gives you the Promise wrapper, not the final value, unless you `await` it.
- `await` pauses execution *inside an async function* until a Promise resolves, then unwraps it to the real value. `await` only works inside a function declared `async`.

Mistake: Calling an `async` function without `await` and using the result directly (`${showNutrition(nutritionSection)}`). **Concept:** Without `await`, you get the Promise object itself, which stringifies to the literal text `"[object Promise]"` when dropped into a template. Fix: `${await showNutrition(nutritionSection)}`, and make sure the enclosing function is `async`.

Mistake: Making a purely synchronous function (`calcPercentage`, just doing math) `async` for no reason. **Concept:** Only mark a function `async` if it genuinely does async work inside (a `fetch`, a real delay). Otherwise it forces unnecessary `await`s everywhere it's called, for zero benefit.

Mistake: `setTimeout(() => { return value }, ms)` inside a function, expecting the outer function to return that value. **Concept:** A `return` inside a `setTimeout` callback only returns to `setTimeout` itself (which ignores it) — it does **not** make the outer function return that value. To actually wait for a delayed value, wrap it in a `new Promise((resolve) => { setTimeout(() => resolve(value), ms) })` and `await` the whole thing.

Concept — guaranteed ordering: Synchronous JS always finishes evaluating one statement (including any `await` inside it) before moving to the next. If value B is built using value A as an input, A is guaranteed to be fully ready before B starts — no extra code needed to "enforce" this order.

---

7. Race conditions

Mistake: A search input with no debounce firing a fetch on every keystroke; a slower response for an earlier partial query can arrive *after* a faster response for the final query, showing stale results. **Concept — Debounce:** delay firing the request until typing pauses (`clearTimeout` + `setTimeout` pattern, ~300ms). Additionally, track the "latest query" and discard any response that no longer matches the current input value, for full protection against out-of-order network responses.

---

8. Template literals

Mistake: Writing `${}` interpolation syntax outside of backticks, as raw function arguments. **Concept:** `${}` only has meaning *inside* a template string (between backticks). To pass a value as a function argument, just reference it directly — no `${}` needed.

Mistake: `<i class=${iconVariable}></i>` — missing quotes around an attribute value that contains a space (e.g. `"fa-solid fa-drumstick-bite"`). **Concept:** Without quotes, an HTML attribute value stops at the first space, silently breaking multi-word classes. Always quote interpolated attribute values: `class="${iconVariable}"`.

---

9. HTML structure inside template strings

Mistake: A missing closing `</div>` deep in a large template shifted everything after it one nesting level too deep, making a "second column" render *inside* the first column instead of beside it. **Concept:** Template literals aren't validated as HTML by the editor — a mismatched tag won't throw a JS error, it just silently produces broken markup. For long templates, indent consistently (matching indentation for open/close tag pairs) so mismatches are easy to spot visually. When something looks structurally wrong, inspect the actual rendered DOM tree in DevTools → Elements to see real nesting.

---

10. Conditional rendering

Concept — optional chaining (`?.`) and nullish coalescing (`??`):

- `obj?.prop` safely returns `undefined` instead of throwing if `obj` is `null`/`undefined`.
- `value ?? fallback` uses `fallback` only when `value` is `null`/`undefined` (unlike `||`, which also replaces falsy-but-valid values like `0` or `""`).

Mistake: Rendering a styled wrapper element (e.g. a colored `<span>` pill) unconditionally, and only making its *inner text* conditional. **Concept:** If the value can be empty, wrap the **entire element** in the ternary, not just its content — otherwise you get an empty-but-still-styled element (e.g. a purple pill with nothing in it).

---

11. Fetch / APIs

Concept — GET vs POST:

- GET: no `method` needed (default), no body — data goes in the URL.
- POST: needs `method: "POST"`, `headers: { "Content-Type": "application/json" }`, and `body:`

`JSON.stringify(data)` — fetch does not serialize objects automatically.

Mistake: Assuming an API endpoint's response shape without checking it first (e.g. assuming `.results` when a single-item endpoint actually returns `.result`, a singular object). **Concept:** Always `console.log` the raw response the first time you use a new endpoint, before writing the code that reads specific keys from it. This is the fastest way to avoid an entire category of `undefined`/crash bugs.

Mistake: Sending the wrong query parameter combo — e.g. combining a free-text search param (`q`) with an unrelated "first letter" or "category" param on the same request, where the backend silently prioritizes one over the other. **Concept:** Different search modes (search-by-name, filter-by-category, browse-by-letter) are usually **separate endpoints**/**params** on an API — don't assume they can be combined without testing the raw URL directly in the browser first.

---

12. Typos are the silent killer

Multiple bugs in this project traced back to a single misspelled property name:

- `.protien` instead of `.protein` → `NaN` width on a progress bar, `undefined` values saved to an object.
- `.vlaue` instead of `.value` → `undefined` passed into a function, crashing deep inside.

Concept: A typo in a property/variable name doesn't throw an error in JS — it just silently returns `undefined`, which can cascade into confusing failures several function calls away from the actual mistake. When something is inexplicably `undefined` or `NaN`, **grep the exact property name** you expect and compare character-by-character against where it's used.

---

13. CSS / Tailwind layout

Concept: `overflow-x-auto` on a flex container makes children scroll horizontally in one line. `flex flex-wrap` makes children wrap onto new lines when they don't fit. These are opposite behaviors — pick based on whether you want scrolling or wrapping.

Concept: `grid-cols-4` (`repeat(4, minmax(0,1fr))`) **never wraps to a second row on its own** — it always keeps 4 columns, just shrinking them if needed. If something that should be a 4-column grid is visually wrapping into 2-per-row, the running code likely doesn't actually match what you think is saved (check for a stale browser cache — hard refresh with `Ctrl+Shift+R` / `Cmd+Shift+R`, or DevTools → "Empty Cache and Hard Reload").

Concept: A grid item that doesn't span the full grid (missing `col-span-full`) will only occupy its first column track, appearing to "not center properly" even though its own internal centering code is correct — the box itself is just narrower than expected.

---

14. Embedding external content

Mistake: Putting a normal YouTube watch URL (`youtube.com/watch?v=...`) directly into an `<iframe src="...">`. **Concept:** YouTube blocks its normal watch pages from being iframed elsewhere (`X-Frame-Options: sameorigin`). Only the `/embed/VIDEO_ID` format is allowed. Extract the video id (regex or URL parsing) and build the embed URL yourself before setting `src`.

---

15. Deep copies

Concept: `{...obj}` or `Object.assign({}, obj)` only copies **top-level** properties — nested objects/arrays are still shared by reference,

so mutating the "copy" can mutate the original too. Use `structuredClone(obj)` for a true deep copy when you need an independent snapshot (e.g. saving a logged meal/product to localStorage without it changing if the source array updates later).

---

16. General debugging workflow that worked well

1. **Read the exact error line/column** — it usually points precisely at the problem, not just "near" it.
2. **Add `console.log` at the boundary of a suspected bug** (before a function call, after a fetch response, inside a loop) rather than guessing blind.
3. **Check the Network tab** to confirm a request is actually firing and what it returns, before assuming the JS logic is wrong.
4. **Check the Elements panel** for actual rendered DOM/CSS when something *looks* wrong but throws no error — this catches structural/nesting bugs that are invisible in the source code view.
5. **When two things run in an unexpected order**, verify with `console.log` in each function rather than assuming — the actual execution order is often already correct, and the issue is elsewhere (e.g. a stale cache, a duplicate call, leftover DOM state).